

openIMIS Hackathon

Participant & Organiser Handbook

A two-day open-source build event on health financing & social protection software

For developers, data scientists, DevSecOps engineers, and UX designers — all experience levels welcome

Duration	Team Size	Challenge Tracks	Beneficiaries Served
2 Days	3 – 5 people	5 Tracks	38.8M+

1. What Is openIMIS?

If you have never worked with openIMIS before, this section is your starting point. Read it carefully before Day 1.

1.1 The Problem openIMIS Solves

More than half the world's population lacks any form of social health protection. In low- and middle-income countries (LMICs), governments, NGOs, and community health schemes struggle to manage the complex flows of data needed to run a health insurance or social protection programme that defines who is enrolled, what benefits they are entitled to, which providers have been paid, and whether the scheme is financially sustainable.

Without reliable software, scheme administrators resort to paper registers, disconnected spreadsheets, and manual reconciliation. This means beneficiaries lose coverage, fraudulent claims go undetected, and governments cannot measure whether their health financing policies are working.

1.2 What openIMIS Is

openIMIS (Open Insurance Management Information System) is free, open-source software that connects three groups of actors in a single platform:

- Beneficiaries — the people enrolled in a scheme (e.g. a rural household covered by a community health fund)
- Providers — hospitals, clinics, pharmacies, and community health workers who deliver care
- Payers — the insurance fund, government body, or NGO that collects contributions and pays providers

By linking all three, openIMIS makes it possible to: enrol members, collect contributions, process and adjudicate claims, pay providers, and analyse scheme performance, all in one system.

Fact	Detail
Founded	2016 — joint initiative of German GIZ and Swiss SDC
Licence	GNU AGPL v3 (fully open source)
Beneficiaries	38.8 million+ people across 14 countries
Active deployments	Tanzania, Nepal, Cameroon, Niger, Mauritania, Zanzibar, and others
Backend	Python 3.10+ / Django / PostgreSQL (or MS SQL)
Frontend	JavaScript / ReactJS
API standard	HL7 FHIR R4 (international health data interoperability standard)
GitHub	github.com/openimis (80+ repositories)
Certified as	Digital Public Good (DPGA registry)

1.3 How openIMIS Is Built: A Gentle Technical Introduction

OpenIMIS has a modular architecture. Think of it like a set of Lego bricks: the core platform provides essential plumbing, and individual modules add specific functionality (claims, beneficiary management, reporting, etc.). Each module lives in its own GitHub repository and registers itself with the core at startup.

There are three main layers:

- Backend (openimis-be_py): A Python/Django application that handles all business logic, database operations, and API endpoints. It exposes data in two ways: a GraphQL API (for the frontend) and a REST/FHIR R4 API (for integration with other systems).
- Frontend (openimis-fe_js): A ReactJS single-page application that provides the web interface for scheme administrators, claims reviewers, and enrolment officers.
- Mobile (openimis-app): A React Native mobile app used by enrolment officers and community health workers in the field, with offline capability.

For the hackathon, you will work primarily with the backend and frontend. The Docker Compose demo stack (openimis-dist_dkr) bundles everything together so you can run a complete local instance in minutes.

1.4 What FHIR Means (and Why It Matters)

HL7 FHIR (Fast Healthcare Interoperability Resources) is the international standard for exchanging healthcare data between systems. Think of it as the 'language' that different health IT systems speak to each other. openIMIS uses FHIR R4 (the current version) so it can talk to other digital health platforms like DHIS2 (health programme analytics) and OpenMRS (electronic medical records).

In FHIR, everything is a 'Resource.' A **Patient** resource describes a person, a **Coverage** resource describes their insurance, and a **Claim** resource describes a service they received. During the hackathon, several challenge tracks require you to work with these resources.

You do NOT need to be a FHIR expert before Day 1. The resources wiki and mentor team will guide you. What matters is understanding that FHIR is the integration 'glue'. If your module needs to exchange data with the outside world, it does so via FHIR resources.

2. Why This Hackathon?

OpenIMIS is used in production by governments and ministries of health to manage real coverage for real people. Every improvement made to the software has the potential to reach millions of beneficiaries. It can also help in better fraud detection, which means more funds are available for patient care; a faster enrolment app means fewer people miss their coverage window.

The openIMIS community is small relative to the impact it has. The hackathon is designed to:

- Bring fresh engineering talent into the openIMIS ecosystem

- Accelerate development of high-priority features that the core team has not had bandwidth to tackle
- Create production-ready contributions that can be merged into the main codebase after the event
- Build a pipeline of developers who continue contributing after the hackathon
- Connect developers with the public health and social protection professionals who use the software daily

3. Who Should Attend?

You do NOT need prior experience with openIMIS. What you do need is:

- Working knowledge of at least one: Python, JavaScript/React, SQL, or data science (scikit-learn / TensorFlow)
- Comfort with Git and GitHub (fork, branch, PR workflow)
- Curiosity about how technology can address public health challenges in LMICs
- Ability to run Docker on your laptop

Ideal participant profiles

Profile	Recommended tracks
Python / Django developer	Tracks 1, 3 — new module development, claims processing, and fraud detection backend
React / frontend developer	Tracks 1, 2 — openIMIS module UI, PHC facility portal, offline-capable enrolment app
Data scientist / ML engineer	Track 3, 5— fraud detection models, AI claims coding, predictive scheme analytics
Mobile / USSD developer	Track 4 — M-PESA integration, USSD payment interface, offline sync engine
Full-stack developer	Any track, particularly Track 1 (open innovation) or Track 4 (mobile payments)
UX / UI designer	Tracks 2, 4 — PHC worker portal, beneficiary payment UX, low-bandwidth accessibility
Public health informatics	Tracks 2, 3, 5 — domain expertise is as valuable as code skills on these tracks

4. Pre-Event Setup (Complete Before Day 1)

Getting your environment working before the hackathon starts is critical. There is no time on Day 1 to debug Docker installs. Follow every step below and verify the checklist at the end.

4.1 Prerequisites

- Git 2.35+ installed
- Docker Desktop (Mac/Windows) or Docker Engine + Compose (Linux) — minimum 8 GB RAM allocated
- Node.js 18+ and npm 9+ (for frontend work)
- Python 3.10+ with pip (for backend module work)
- A GitHub account
- A code editor (VS Code recommended — install the Python and ESLint extensions)

4.2 Step-by-Step: Running openIMIS Locally

Step 1 — Fork the repositories

Go to github.com/openimis and fork the following repositories into your own GitHub account:

- `openimis-dist_dkr` (the Docker demo stack — start here)
- `openimis-be_py` (Python/Django backend — needed for backend tracks)
- `openimis-fe_js` (ReactJS frontend — needed for frontend tracks)

Step 2 — Clone and start the Docker stack

```
git clone https://github.com/YOUR_USERNAME/openimis-dist_dkr
cd openimis-dist_dkr
docker-compose up -d
```

This will download and start all openIMIS containers. The first run takes 5–10 minutes. Subsequent starts are fast.

Step 3 — Verify the installation

- Open `http://localhost` in your browser — you should see the openIMIS login screen
- Open `http://localhost:8000/graphql` — you should see the GraphQL API explorer
- Default admin credentials: Username `admin` / Password `admin` (change immediately on a production system)

Step 4 — Load the demo dataset

openIMIS provides an anonymised demo database based on Tanzania scheme data. This gives you realistic beneficiaries, claims, and providers to work with.

```
docker-compose exec db psql -U postgres -d openimis -f /demo-data/demo.sql
```

(Exact command may vary — the hackathon organiser will provide the specific demo dataset and load instructions in the participant Discord/Slack channel before the event.)

Step 5 — Join the hackathon channels

- Discord/Slack: Join the event Discord server using the invite link in your confirmation email
- GitHub: Join the hackathon organisation team to access shared resources
- Test your setup: Post a screenshot of your running openIMIS instance in the **#setup-help** channel

4.3 Setup Verification Checklist

☐	Docker stack starts without errors (docker-compose up -d)
☐	openIMIS UI loads at http://localhost
☐	Can log in with admin/admin credentials
☐	GraphQL explorer accessible at http://localhost:8000/graphql
☐	Demo dataset loaded — at least 10 insurees visible in the Insurees module
☐	GitHub fork created for all three repositories
☐	Joined the hackathon Discord posted in #introductions
☐	VS Code (or preferred IDE) configured with Python and ESLint extensions

If you get stuck, post in **#setup-help** on Discord. Mentors are available for setup support from 5 days before the event.

5. Event Schedule

DAY 1 — DISCOVER, DESIGN & BUILD

07:30 – 08:30

Registration & Environment Check

Arrive, check in, collect your starter pack, and verify your Docker stack is running. Tech volunteers available for last-minute setup issues.

08:30 – 09:15

Opening Keynote: openIMIS in the Real World

An introduction to openIMIS — what it is, who uses it, and where the community needs help. A guest speaker from an active deployment country shares field experience.

09:15 – 09:45

Technical Deep-Dive Session

A core openIMIS contributor walks through: the Django module system, how to add a new module, the GraphQL API, FHIR R4 endpoints, and React component patterns. Slides shared on Discord.

09:45 – 10:00

Challenge Reveal & Team Formation

Five challenge tracks announced. Teams of 3–5 people self-select a track. Register your team name and GitHub fork URL using the provided form.

10:00 – 10:20

Coffee Break & Mentor Introductions

Meet your track mentors. Each mentor covers 2–3 teams and is available throughout the day.

10:20 – 13:00

Sprint 1: Architecture & Core Data Models

Define your team's scope. Produce an architecture diagram. Set up your GitHub project board. Implement core Django models or React component skeletons. Mentors rotate every 45 minutes.

13:00 – 14:00

Lunch + Optional Lightning Talks

30-minute catered lunch, then two 15-minute expert talks: (1) FHIR interoperability patterns in openIMIS; (2) How the AI claims adjudication module works.

14:00 – 15:50

Sprint 2: Core Feature Build

Main coding block. Focus on implementing the primary feature of your chosen track.

16:00 – 16:20

Mid-Sprint Check-In

Each team gives a 2-minute status update. Mentors help identify blockers and trim scope where needed.

16:20 – 17:30	Sprint 2 Continues <i>Back to coding. Push a well-labelled commit before 17:30.</i>
17:30 – 18:00	Day 1 Stand-Down & Commit <i>Push a tagged commit (v0.1-hackathon) to your fork. Brief plenary on overnight goals. Mentors summarise common blockers.</i>
18:00 – 20:00	Optional Evening Session (Remote) <i>Async collaboration on Discord. Mentors available for 1:1 video debugging calls until 20:00.</i>

DAY 2 — INTEGRATE, POLISH & DEMO

08:00 – 09:00	Morning Stand-Up & Re-Planning <i>Each team: 3-minute update covering progress, blockers, and revised Day 2 scope. Mentors help teams cut to a shippable MVP if needed.</i>
09:00 – 11:30	Sprint 3: Integration & Testing <i>Wire up API endpoints. Write unit tests. Integrate with openIMIS core modules. Tighten UX. Begin documentation.</i>
11:30 – 12:00	Demo Preparation & README <i>Stop coding. Write your README (problem statement, installation steps, architecture overview, screenshots). Prepare your 8-minute demo script.</i>
12:00	CODE FREEZE <i>All code committed and tagged (v1.0-hackathon). No further pushes to the submission branch after this time.</i>
12:00 – 13:00	Lunch <i>Judges access repositories and begin scoring. Judge scorecards distributed.</i>
13:00 – 15:30	Live Demo Presentations <i>Each team: 8 minutes live demo + 4 minutes judge Q&A. Presentation order drawn randomly. Public audience welcome.</i>
15:30 – 16:15	Judge Deliberation <i>Judges score entries in private. Community vote tallied for People's Choice award. Refreshments served.</i>
16:15 – 17:00	Awards & Closing Ceremony

Winners announced across five award categories. Keynote on open-source contribution pathways — how your hackathon code gets into production. Networking and celebration.

6. Challenge Tracks

Choose one track per team. Each track is designed to produce a meaningful, shippable contribution to the openIMIS ecosystem. You do not need to complete every deliverable; a focused, well-executed MVP scores higher than a sprawling half-finished feature.

Track 1 : openIMIS Innovation Track

Challenge statement

Build solutions, extensions, or integrations that enhance the functionality and adoption of openIMIS. This is the broadest and most flexible track. It is deliberately open-ended so that teams can address any gap in the openIMIS ecosystem that they identify as genuinely important.

Context for first-time participants

If you are new to openIMIS, this is the recommended starting track. The openIMIS module system is designed so that new features can be added as independent Django/React packages without modifying core code. By the end of Day 1 Sprint 1, your goal is to have a working skeleton module registered with the platform, even if it only exposes one endpoint and one React page.

Example solution ideas

- A beneficiary grievance and appeals module — allowing insurees to dispute claim rejections through a structured workflow
- A provider credentialing module — tracking which facilities are accredited and flagging expired credentials at claim submission
- A multi-language support layer — making openIMIS UI strings translatable and shipping Swahili, French, or Amharic language packs
- A bulk data import tool — allowing scheme administrators to onboard thousands of beneficiaries via a validated CSV or Excel upload
- A DHIS2 or OpenMRS integration bridge — exposing openIMIS data as FHIR R4 resources consumable by external health platforms
- A scheme comparison dashboard — side-by-side analytics for multi-scheme deployments

Minimum viable submission

- A new openIMIS backend module (Django) registered via `openimis.json` with at least one model, one GraphQL query, and one REST endpoint
- A corresponding frontend module (React) with at least one meaningful page accessible from the openIMIS navigation menu
- Unit tests covering core business logic
- A README explaining the problem solved, the architecture, and how to install the module

Bonus deliverables

- FHIR R4 resource exposure for any new data models introduced
- A Docker Compose override file that adds the module to the demo stack in one command

Skills you will use

- Python / Django / PostgreSQL, ReactJS, GraphQL (Graphene-Django), REST APIs, FHIR R4 (optional)

Track 2 : Digital Solutions for Primary Healthcare Financing

Challenge statement

Strengthen financing mechanisms that support primary healthcare facilities and community health services. Primary healthcare (PHC) is the first point of contact for most patients. Yet, facilities in LMICs are chronically underfunded, partly because they lack digital tools to submit claims accurately, track capitation payments, or report on service utilisation.

Context for first-time participants

In openIMIS, a 'benefit package' defines which services are covered and at what price. A 'health facility' (provider) submits claims against these packages. Community health workers (CHWs) enrol beneficiaries in the field. The problem this track addresses is that PHC facilities are the lowest level of the health care system. These facilities often cannot engage with openIMIS because the tools are too complex, require constant connectivity, or do not model PHC-specific financing mechanisms like capitation or output-based aid.

Example solution ideas

- A capitation payment module — calculating and disbursing fixed per-head monthly payments to PHC facilities based on their enrolled population
- An output-based aid (OBA) tracker — recording service delivery against pre-defined targets and triggering conditional payments on achievement
- A community health worker (CHW) portal — a simplified, offline-capable interface for CHWs to record service visits, referrals, and contribution collections
- A PHC facility performance scorecard — visualising quality indicators (utilisation rate, claim rejection rate, average turnaround) for facility managers
- An equipment and supplies linkage module — connecting PHC financing flows to supply chain data to ensure funded services can actually be delivered

Minimum viable submission

- A new openIMIS module that models at least one PHC-specific financing mechanism (capitation, OBA, or CHW performance pay) with correct Django models and GraphQL API
- A React interface usable by a non-technical PHC facility administrator — minimal jargon, clear status indicators
- Offline capability for at least one core workflow (enrolment, service recording, or contribution collection)
- A working end-to-end scenario demonstrated with the demo dataset

Bonus deliverables

- Integration with the openIMIS location hierarchy — payments and reports filtered by ward / sub-county / county
- Automated SMS or push notification to CHWs when a payment is processed (using Africa's Talking API or similar)

Skills you will use

- Python / Django, React Native or PWA, REST/FHIR APIs, offline-first architecture (IndexedDB, service workers)

Track 3 : Claims Management and Fraud Detection

Challenge statement

Improve the efficiency, transparency, and accuracy of health insurance claims processing. As the operational heart of any health insurance scheme, claims adjudication is where coverage policies meet real-world service delivery. However, the current process faces critical bottlenecks: Slow processing delays provider payments, which erodes trust; fraudulent or duplicate claims drain scheme funds that should reach patients.

Context for first-time participants

In openIMIS, a Claim is submitted by a health facility on behalf of a patient. It contains the patient ID, the services provided (mapped to a benefit package), the amounts claimed, and the visit date. A claims reviewer then adjudicates the claim by approving, partially approving, or rejecting it. The FHIR R4 equivalents are the *Claim* and *ClaimResponse* resources. This track asks you to make adjudication faster, smarter, and more transparent.

Example solution ideas

- An AI fraud detection engine — a trained ML model that scores each claim for fraud risk before it reaches a human reviewer, using features like claim frequency, amount deviation from benchmark, and provider behaviour patterns
- A duplicate claim detector — an algorithm that identifies claims submitted multiple times for the same patient visit, using fuzzy matching on date, facility, and service codes
- An automated pre-adjudication rules engine — a configurable rules layer that auto-approves low-risk claims meeting all criteria and flags only exceptions for human review
- A claims audit trail dashboard — an immutable log of every status change on a claim, queryable by auditors and exportable for regulatory reporting
- A provider feedback portal — allowing health facilities to view claim statuses, receive structured rejection reasons, and resubmit corrected claims

Minimum viable submission

- A working claims scoring or filtering mechanism integrated into the openIMIS claims workflow via a Django signal or management command
- FHIR R4 Claim Response resources correctly populated with adjudication outcome and any extension fields added by your module
- A React UI component that surfaces the scoring output to a claims reviewer within the existing claims list view
- A performance report (if ML is used): precision, recall, F1 score, and confusion matrix on the demo dataset

Bonus deliverables

- A feedback loop — when a reviewer overrides the model decision, that action re-enters the training pipeline
- A real-time claims throughput metric visible on the administrator dashboard

Skills you will use

- Python, scikit-learn or TensorFlow (for ML approaches), Django, GraphQL, React, FHIR R4 ClaimResponse

Track 4: Mobile Health Payments and Financial Inclusion

Challenge statement

Increase access to healthcare financing through mobile and digital payment solutions. In East and West Africa, mobile money (M-PESA, MTN MoMo, Airtel Money) has transformed how payments and contributions are made, yet most health insurance schemes still collect contributions through cash and paper receipts. The result is that millions of eligible people miss their coverage window simply because the payment process is inaccessible or unreliable.

Context for first-time participants

In openIMIS, a Contribution is a payment made by or on behalf of a beneficiary to maintain their coverage. The current model supports manual entry of contributions by an enrolment officer. This track asks you to build a bridge between mobile money infrastructure and the openIMIS contributions module, so that a beneficiary can pay their premium via M-PESA and see their coverage updated in near-real-time, without any paper or intermediary.

Example solution ideas

- An M-PESA STK Push integration — beneficiaries pay premiums from their mobile phone; the payment confirmation webhook updates their Coverage record in openIMIS automatically
- A USSD-based enrolment and payment interface — enabling feature-phone users (no smartphone required) to check coverage status and pay contributions via *XXX# short codes
- A digital contribution receipt and coverage certificate — automatically generated and sent via SMS or WhatsApp when a contribution is confirmed, with a QR code for facility verification
- A group payment gateway — allowing a household head or village savings group to pay premiums for multiple family members in a single mobile transaction
- A contribution arrears notification system — proactively alerting beneficiaries approaching lapse via SMS, with a one-tap payment link

Minimum viable submission

- A working payment gateway integration (M-PESA Daraja API, or a credible sandbox/mock if live credentials are unavailable) that creates a Contribution record in openIMIS on payment confirmation
- A webhook receiver endpoint (Django) that processes payment callbacks securely and idempotently
- A React or USSD interface through which a beneficiary or officer can initiate a payment
- End-to-end test: payment initiated → callback received → Coverage status updated → confirmation sent

Bonus deliverables

- Support for multiple mobile money providers via an abstraction layer (strategy pattern)
- A reconciliation report comparing mobile money settlement statements to openIMIS contribution records

Skills you will use

- Python / Django, REST APIs, M-PESA Daraja API (or equivalent), React or USSD framework (Africa's Talking), webhook security (HMAC verification)

Track 5: AI and Emerging Technologies for Health Financing

Challenge statement

Leverage emerging technologies to improve efficiency and decision-making in health financing. This track has the widest technology aperture they include; AI, machine learning, large language models, computer vision, voice interfaces, federated learning, or blockchain-based transparency. The constraint is that your technology must demonstrably improve an outcome in health financing, not just be technically impressive.

Context for first-time participants

Health financing in LMICs generates rich but messy data: handwritten claim forms, scanned receipts, free-text diagnosis codes, fragmented patient records across facilities. Emerging technologies offer real leverage here, but the graveyard of 'AI for health' pilots is full of models that performed well in notebooks and failed in production. This track rewards solutions that are technically sound AND practically deployable in a low-resource environment.

Example solution ideas

- An LLM-powered claims coding assistant — a chatbot that helps facility coders assign the correct ICD-10 and procedure codes from free-text clinical notes, reducing coding errors that cause claim rejections
- A computer vision receipt verifier — using OCR and image classification to extract and validate line items from photographed paper claims before they enter openIMIS
- A natural language policy explainer — an LLM that answers beneficiary questions about their coverage in plain language and in a local language, grounded in the scheme's actual benefit package data
- A federated learning model for fraud detection — training a shared fraud detection model across multiple scheme deployments without sharing raw patient data between countries
- A voice-based enrolment agent — a voice UI using STT/TTS that walks a CHW through the beneficiary enrolment form in a local language, populating openIMIS via API
- A predictive scheme viability model — forecasting contribution income, claims expenditure, and reserve adequacy 6–12 months ahead to support management decisions

Minimum viable submission

- A working proof-of-concept integrated with openIMIS via API — not a standalone demo disconnected from the platform
- Quantified improvement: a before/after comparison showing how your AI feature changes a measurable outcome (coding accuracy, adjudication speed, enrolment completion rate, or fraud detection rate)
- A responsible AI section in your README: what data was used for training, what biases were considered, and how the model output is explained to end users
- An inference endpoint (REST or GraphQL) that the openIMIS frontend can call in real time

Bonus deliverables

-
- A model card documenting training data, performance metrics, known failure modes, and recommended use conditions
 - Multi-language support (Swahili, French, or another relevant language) for any NLP or voice component

Skills you will use

- Python, scikit-learn / TensorFlow / PyTorch, HuggingFace Transformers (for LLM work), OpenCV or Tesseract (for vision/OCR work), Django, REST API integration

7. Judging Criteria & Awards

7.1 How Scoring Works

Eight Independent judges score each team using the rubric below. Scores are averaged. A fourth score, “*People's Choice*,” is voted by all participants using a separate ballot. The two scores are not combined;

Demo format: 8 minutes live demo (no pre-recorded video) + 4 minutes judge Q&A. A working demo is required, while **slide-only** presentations are **not accepted**.

Criterion	What judges look for	Points
Technical implementation	Code quality, architecture decisions, use of openIMIS module patterns, test coverage. Does it run without errors?	30 / 100
openIMIS integration	Correct use of FHIR R4 resources, GraphQL API, Django module registration. No monkey-patching of core openIMIS code.	25 / 100
Real-world impact	Does this solve a genuine pain point for a scheme operator, beneficiary, or payer in an LMIC context? Would a real user want this?	20 / 100
Demo & presentation	Live working demo, clear problem statement, articulate answers to judge’s questions.	15 / 100
Documentation	README with setup instructions, inline docstrings or JSDoc, open PR or issue on openIMIS GitHub.	10 / 100

7.2 Award Categories

- A certificate and badge will be awarded to all teams that participate in the hackathon.

7.3 Judging Panel

- One representative from a live openIMIS deployment (scheme operator or Ministry of Health official).
- At least eight technical Engineers from Strathmore University iLabAfrica.
- One senior engineer, architect, or implementer from a partner organization (KHIA, GIZ, SDC, Digital Square, or WHO)

8. Rules & Code of Conduct

8.1 Submission Rules

- All code must be committed to a public GitHub fork of the relevant openIMIS repository before the code freeze at 12:00 on Day 2.
- Tag your submission commit as v1.0-hackathon.
- Your README must include: problem statement, architecture overview, installation steps, and known limitations.
- You must open a draft PR or issue on the official openIMIS GitHub repository pointing to your fork. This is required for the Documentation criterion.
- All submitted code must be licensed under GNU AGPL v3, consistent with openIMIS.
- You may use third-party libraries and AI coding assistants, but the architecture decisions and integration logic must be your team's own work.

8.2 Code of Conduct

This event follows the Contributor Covenant Code of Conduct. In summary:

- Treat all participants, mentors, judges, and organisers with respect.
- No harassment, discrimination, or exclusionary behaviour of any kind.
- Constructive critique of code is welcome; personal attacks are not.
- Violations should be reported to the event organiser immediately. Organisers reserve the right to remove participants for code of conduct violations.

9. Resources & Reference Links

9.1 Essential Links

Resource	URL / Description
openIMIS homepage	https://openimis.org
GitHub organisation	https://github.com/openimis (80+ repositories)
Developer wiki (Confluence)	https://openimis.atlassian.net/wiki/spaces/OP
Docker demo stack	github.com/openimis/openimis-dist_dkr
Backend repo	github.com/openimis/openimis-be_py
Frontend repo	github.com/openimis/openimis-fe_js
FHIR R4 API module (PyPI)	<code>pip install openimis-be-api-fhir-r4</code>
HL7 FHIR R4 specification	https://hl7.org/fhir/R4
Graphene-Django (GraphQL)	https://docs.graphene-python.org/projects/django
openIMIS community forum	https://community.openimis.org

9.2 Recommended Reading (Before the Event)

- openIMIS modular architecture overview — Developer wiki, 'Architecture' section
- HL7 FHIR R4 core resources: Patient, Coverage, Claim, ClaimResponse — hl7.org/fhir/R4
- Django REST framework documentation — [django-rest-framework.org](https://www.django-rest-framework.org)
- Graphene-Django tutorial — for GraphQL extension work
- Digital Square openIMIS case studies — digitalsquare.org (Tanzania, Nepal, Cameroon)
- WHO digital health investment guide for LMICs — who.int/publications

9.3 Getting Help During the Event

- In person: Find your track mentor — each wears a coloured lanyard matching their track
- Discord **#help-general**: For questions not specific to a track
- Discord **#help-track-1** through **#help-track-5**: Track-specific mentor channels
- GitHub Issues: Open an issue on the hackathon organisation repo for persistent bugs

10. Glossary for First-Time openIMIS Participants

If you are new to openIMIS or health informatics, these terms appear frequently:

Term	Plain-language meaning
Insuree	A person enrolled in a health insurance scheme — essentially a 'member' or 'beneficiary'
Payer	The organisation (government body, insurance fund, NGO) that collects contributions and pays for services
Provider	A hospital, clinic, pharmacy, or community health worker that delivers care to insurees
Claim	A formal request for payment submitted by a provider after treating an insuree (e.g. hospital submits a claim for a surgery)
Adjudication	The process of reviewing a claim and deciding whether to approve, partially approve, or reject it
Contribution	A premium or fee paid by or on behalf of a beneficiary to maintain their coverage
Benefit package	The specific set of health services covered by an insurance scheme for its members
FHIR	Fast Healthcare Interoperability Resources — the international standard for exchanging health data between software systems
FHIR Resource	A structured data object in the FHIR standard — e.g. Patient, Coverage, Claim are all 'resources'
HL7	Health Level Seven International — the standards body that created FHIR
DHIS2	District Health Information System 2 — open-source health analytics platform used by 70+ countries
OpenMRS	Open Medical Record System — open-source electronic health record platform for LMICs
OpenHIM	Open Health Information Mediator — an integration engine used as a gateway between health systems
GraphQL	A query language for APIs; openIMIS uses it for frontend ↔ backend communication
LMIC	Low- and Middle-Income Country — the primary context for openIMIS deployments
DPG	Digital Public Good — a designation confirming open-source tools meet standards for development impact
AGPL v3	GNU Affero General Public Licence version 3 — the licence under which openIMIS is released

Ready to build something that matters?

openIMIS is used by 38.8 million real people. Your two days of work could improve care access for communities across Africa and Asia.

Questions? Post in #general on the hackathon Discord or email the organising team.